

MergeNet: Spatial Routing Gains and Deployment Tradeoffs in Learned Token Compression

Anonymous authors

Abstract

Spatial constraints can improve learned token aggregation without making the resulting model an efficient alternative to dense computation. We examine this distinction in MergeNet, a DTEM-derived classifier that performs original-grid-constrained soft mass transfer, selects a fixed number of carriers, and recovers information before latent processing. In a single-seed ImageNet-1k campaign, radius-3 routing improves 150-epoch top-1 accuracy by 1.670 percentage points over an unconstrained router and 0.598 points over a flattened-window control. The full model reaches 81.368% after 300 epochs at 224 pixels, versus 82.246% for dense DeiT-S/8; 384-pixel fine-tuning narrows the gap to 82.582% versus 82.976%. Despite 36.9% fewer estimated multiply-accumulates, MergeNet takes 7.1% more inference time in a guarded 224-pixel synthetic benchmark, while a 384-pixel batch-64 benchmark reduces inference time by 20.0%. Post-training ToMe has higher recorded accuracy at all twenty matched final-token budgets, although incompatible attention settings prevent combining the available accuracy and latency results into a verified joint frontier. An operator-level probe additionally reveals batch-dependent soft selection. The study separates internal routing gains from deployment evidence and identifies the implementation and reproduction checks needed to assess learned compression reliably.

1 Introduction

Fine patch grids preserve spatial detail but increase the cost of vision transformers. Token merging offers a way to retain information from that grid while reducing the sequence processed by later layers [1]. Learning the merge criterion can make compression responsive to the task, but a useful learned router must satisfy two distinct requirements: its aggregation decisions must preserve useful representations, and its execution must save enough work to pay for routing itself. An improvement in the first requirement does not imply an improvement in the second.

We study this distinction through MergeNet, a classifier that combines a DTEM-derived soft router [6] with a full-grid local encoder, a fixed carrier budget, global recovery attention, and a compressed latent encoder. The routing constraint is simple: a donor can send mass only to receiver slots within a Euclidean radius on the original image grid. The constraint prevents a flattened sequence from treating a row boundary as ordinary spatial adjacency. It also limits candidate support independently of the learned similarity. We compare this router with unconstrained and flattened-window versions of the same architecture before comparing the complete model with dense and post-training compression baselines.

The resulting ImageNet-1k campaign supports a qualified conclusion. With matched 150-epoch settings, radius-3 routing improves best validation accuracy by 1.670 *pp* over the unconstrained router and 0.598 *pp* over the flattened window. These are observed single-seed improvements, rather than estimates of a population effect. They do not establish that geometry alone explains the benefit: the constraints also change candidate counts and the score normalization context.

The deployment evidence is less favorable. At 224 pixels, a 300-epoch MergeNet reaches 81.368% top-1, compared with 82.246% for dense DeiT-S/8. Its batch-64 inference takes 45.71 ms on a guarded H20 benchmark, compared with 42.68 ms for the dense model, despite lower estimated arithmetic cost. After 384-pixel fine-tuning, the accuracy gap narrows to 0.394 *pp*. Post-training ToMe remains a strong accuracy baseline at matched final patch counts. However, the available ToMe accuracy and latency harnesses use different proportional-attention settings, so their outputs cannot be combined into a verified accuracy-latency frontier.

Our contributions are an implementation-specific analysis of spatially constrained soft routing, a traceable evaluation of its internal and external comparisons, and a diagnosis of why nominal compression and realized savings can diverge. We explicitly distinguish the support of a routing step from the receptive

field of a feature, and soft mass transfer from physical sequence reduction. The study establishes useful internal controls and concrete reproduction requirements; it does not claim state-of-the-art compression, priority for spatial merging, or a general failure of learned token merging.

2 Related Work

Token reduction methods differ in which tokens they retain, how they aggregate information, and where they reduce computation. DynamicViT learns token retention decisions and trains them using differentiable attention masking [10], while A-ViT assigns adaptive computation depth through token halting [17]. Token Merging (ToMe) combines similar tokens through bipartite soft matching and can be applied to an existing checkpoint without additional training or incorporated into training [1]. Subsequent work changes the merge criterion: PiToMe protects informative or isolated tokens using an energy score [13]; MCTF combines similarity, informativeness, and token size with one-step-ahead attention and reduction-consistency training [7]; SAD-TM uses visual saliency and a deferred-merging variant [15]. These methods provide stronger comparisons for a compression mechanism than an uncompressed backbone alone.

The closest methodological predecessor of MergeNet is Decoupled Token Embedding for Merging (DTEM), which learns low-dimensional merge embeddings through continuously relaxed grouping and merging [6]. DTEM separates the representation used to select merges from the features used by the backbone. Its soft embedding updates retain the physical token count; end-to-end training alternates these updates with backbone updates that use hard merging, and inference uses bipartite matching. MergeNet builds on this lineage through the OpenToMe implementation [14]. Our study concerns the support of the routing operator: which donor–receiver pairs are eligible under a prescribed carrier budget. The decoupled embedding and soft merge operators are inherited components. The unconstrained MergeNet variant is an internal architectural control and should not be interpreted as a reproduction of the original DTEM training and inference protocol.

Spatial locality is also an established design choice for token reduction. DSM delays merging, applies it within spatial windows, and then transitions to global merging [4]. ALGM similarly combines early local merging with later global merging for semantic segmentation [9]. GTP-ViT propagates tokens over a graph that combines adjacency on the original image grid with semantic connections [16]. More recent spatial reducers impose different constraints: CubistMerge restricts matching to neighbors along rows and columns while preserving a rectangular token layout [3], whereas MAT merges adjacent tokens after Hilbert-curve reordering [8]. MergeNet instead defines eligible routes by a Euclidean radius on the original patch grid. This distinction motivates controlled comparisons of global, flattened-window, and two-dimensional routing support; it does not establish priority for locality or for hard spatial constraints.

Compression and recovery must also be distinguished from the routing rule. LookupViT processes a fixed set of compressed tokens alongside a cheaper high-resolution token stream, exchanging information through bidirectional cross-attention [5]. ALGM restores token resolution for dense prediction [9], while DToP uses early predictions for confident segmentation tokens and combines predictions across stages without reconstructing those tokens [11]. These methods show that retaining or recovering dense information is a separate architectural choice. Our classification study evaluates the behavior and cost of a particular local–latent architecture; it does not establish a new general recovery mechanism or its suitability for dense prediction.

Evaluation protocols determine what a comparison can establish. ToMe includes DeiT models trained from scratch for 300 epochs, whereas DTEM primarily studies adaptation of pretrained models [1, 6]. Published results also depend on the backbone and its training recipe [12]; they cannot substitute for controlled comparisons in our patch-8 setting. DiffRate further demonstrates that layerwise compression rates are a design variable distinct from the merge criterion [2]. Equal final token counts therefore do not imply equal computation throughout the network. Moreover, DSM reports that data movement associated with local merging can limit throughput gains despite arithmetic savings [4]. We accordingly separate the internal geometry study from comparisons with post-training ToMe and PiToMe, and distinguish achieved token counts, arithmetic cost, and measured latency. The resulting scope is a study of spatial routing and its execution costs, with conclusions bounded by the evaluated models, training protocols, and hardware.

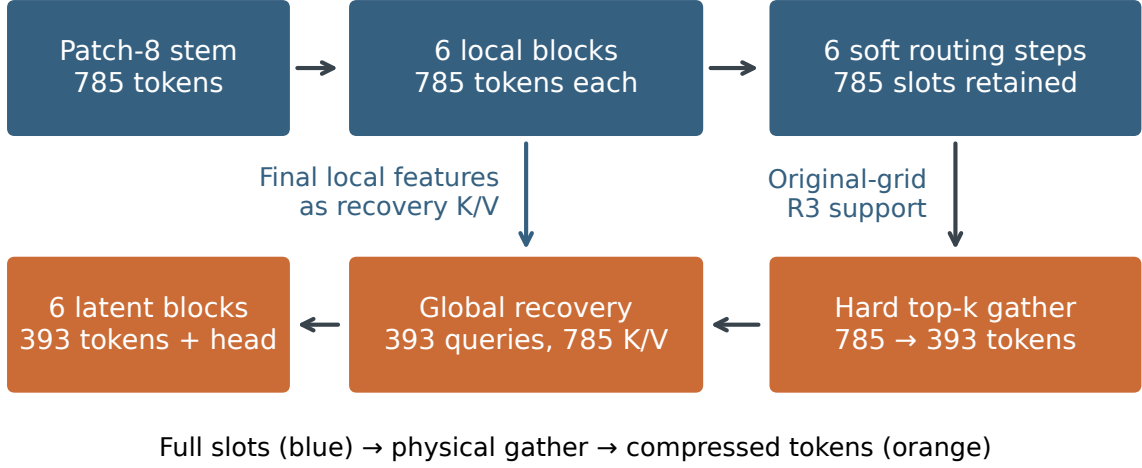


Figure 1: Main MergeNet dataflow at 224 pixels. All six local blocks run before routing; every soft routing step retains 785 slots. The hard gather is the first physical reduction. Recovery reads the final local features before soft transport. Blue marks full-grid processing and orange marks compressed processing. Numbers include the class token.

3 Spatially Constrained Soft Routing

3.1 Architecture and actual sequence reduction

For an image of size $H \times W$ and patch size p , let $N = (H/p)(W/p)$ denote the number of patch slots, excluding a protected class token. The main configuration uses $p = 8$, width $d = 384$, six attention heads, six local blocks, and six latent blocks. At 224×224 , the local encoder processes $N + 1 = 785$ tokens in every block. Its attention window is a one-dimensional band extending 16 sequence positions in each direction, with global attention for the class query. It is not a 16×16 spatial window. Two-dimensional geometry enters through the routing support below.

The local encoder first computes all six layer outputs, $H^{(1)}, \dots, H^{(L)}$. Routing then starts from the last output, $X^{(0)} = H^{(L)}$, using a separate metric derived from each $H^{(\ell)}$. Each routing step retains the complete slot tensor. Only after these steps are completed does a hard gather physically reduce the sequence. This order is essential: earlier local blocks do not benefit from progressively shorter sequences. Figure 1 depicts the executed dataflow.

3.2 Original-grid eligibility

Index patch slots by $i \in \{0, \dots, N - 1\}$ and define original coordinates

$$u_i = (\lfloor i/(W/p) \rfloor, i \bmod (W/p)). \quad (1)$$

At step ℓ , patches are partitioned into donor and receiver sets A_ℓ and B_ℓ . Training uses a random partition for each sample; evaluation alternates deterministic odd/even partitions across steps. The class token belongs to neither merging set. For donor i , eligible receivers are

$$\mathcal{E}_i^{(\ell)}(R) = \{j \in B_\ell : \|u_i - u_j\|_2 \leq R\}. \quad (2)$$

Coordinates refer to the original slots throughout routing. They are not replaced by the centers of the accumulated feature content. The main choice is $R = 3$ patch spacings. The global control removes this spatial constraint; the flattened control uses the release’s one-dimensional matching window with width parameter 8.

Let $z_i^{(\ell)}$ be the normalized output of a learned $384 \rightarrow 64$ metric projection applied to $H_i^{(\ell)}$. Its input gradient is scaled by 0.1 without changing its forward value. Following the inherited relaxed-routing construction, eligible similarities define

$$s_{ij} = \langle z_i^{(\ell)}, z_j^{(\ell)} \rangle / \tau_1, \quad (3)$$

$$e_i = \sum_{j \in \mathcal{E}_i^{(\ell)}} \exp(s_{ij}), \quad q_{ij} = \frac{\exp(s_{ij})}{e_i} \quad (j \in \mathcal{E}_i^{(\ell)}). \quad (4)$$

A threshold-based soft top- k operator converts row energies e_i into donor weights g_i , with the step’s scheduled merge budget as its target. We use $\tau_1 = 1$ and multiplicative soft-selection scale 30. The released threshold routine uses a batch-wide range normalization (Section 6.2); its requested budget does not guarantee exact relaxed selection mass for every numerical input. The assignment is $a_{ij} = g_i q_{ij}$ on eligible edges and zero elsewhere, with row-sum stabilization in the implementation. Empty rows have zero assignment. If fewer eligible donor rows exist than the requested budget, the spatial path raises an error instead of routing outside the support. Our notation suppresses the implementation’s numerical clamps and straight-through stabilizers; it does not assume the relaxed weights are exact binary choices.

3.3 Mass transfer and fixed-budget selection

Each slot carries a nonnegative mass m_i , initialized to one. For a donor i and a receiver j , soft transport updates masses and receiver features as

$$m'_i = m_i \left(1 - \sum_j a_{ij} \right), \quad X'_i = X_i, \quad (5)$$

$$m'_j = m_j + \sum_i a_{ij} m_i, \quad (6)$$

$$X'_j = \frac{m_j X_j + \sum_i a_{ij} m_i X_i}{m'_j}. \quad (7)$$

The code clamps residual donor fractions and receiver denominators for numerical stability. Donor slots remain in the tensor even when their mass is small. Thus soft transport and token deletion are different operations. The main spatial path evaluates a sparse neighbor list and uses indexed transport; a dense score-then-mask path is available as a reference.

For compression parameter λ , the final number of patch carriers is

$$K = N - \left\lfloor N \frac{\lambda - 1}{\lambda} \right\rfloor. \quad (8)$$

The model gathers the K largest-mass patches and prepends the protected class token. At $\lambda = 2$, $K = 392$ at 224 pixels and $K = 1152$ at 384 pixels. At $\lambda = 2$, the six soft donor-budget targets are $(65, 65, 65, 65, 65, 67)$ at 224 pixels and $(192, 192, 192, 192, 192, 192)$ at 384. The carrier count is fixed by configuration; the selected locations depend on the input. A soft top- K surrogate supplies straight-through gradients through the selected features via $1 + g - \text{stopgrad}(g)$; it does not differentiate every gather index. This multiplier is computed at evaluation too and equals one for finite values, whereas non-finite values do not cancel. During training an auxiliary head contribution is added to the main logits, with maximum weight 0.05; it is absent during evaluation. This is a logit contribution, not a separately reported auxiliary cross-entropy loss.

The default source-tracing mode maintains a detached mass-weighted center of the flattened indices. It uses these values to sort selected carriers; it does not retain the full source-membership matrix. Selected features Q then undergo residual recovery,

$$Y = Q + \text{CrossAttn}(Q, H^{(L)}, H^{(L)}). \quad (9)$$

Recovery is global in this mode. Six global latent blocks process $K + 1$ tokens, with no further merging. Latent attention retains proportional attention: it adds the logarithm of the selected token mass to key logits before the softmax. The class representation feeds the classification head.

3.4 What the geometry and complexity claims mean

Equation (2) bounds a single transport edge. It does not bound the spatial extent of all original constituents after multiple steps: a sequence of locally permitted transfers can travel farther. Moreover, $H^{(L)}$ already contains contextual information and Equation (9) reads globally. The constraint therefore concerns routing support, not the entire model’s receptive field or semantic segmentation boundaries.

The distinction between slots and carriers also determines cost. Local attention projections and MLPs still process $N + 1$ tokens per layer. Recovery has attention terms proportional to NKd , and latent self-attention has terms proportional to K^2d . For fixed λ , both retain quadratic terms in N . Sparse routing avoids a dense donor–receiver assignment during the forward pass, but initialization still constructs an $N \times N$ Boolean adjacency buffer. Neither a linear-memory claim for the complete implementation nor an end-to-end speedup follows from the sparse scorer alone.

4 Experimental Protocol

4.1 Data, optimization, and run identity

We analyze an ImageNet-1k classification campaign with seed 42. Evaluation uses the 50,000-image validation split and reports top-1 accuracy of the exponential moving average (EMA) model. Every available training summary row was matched to its final per-epoch EMA validation line in the launcher log: 2,419 rows across 17 runs with recorded epochs. There are 18 launched runs in total: 16 complete schedules, one partial curriculum snapshot, and one 512-pixel MergeNet run with no completed epoch in the supplied record. Checkpoint-verification receipts are present, but the weights themselves were not transferred with the campaign packet. This audit verifies the recorded outcomes; it is not an independent checkpoint re-evaluation.

The 224-pixel runs train from scratch for 150 or 300 epochs, using AdamW, cosine learning-rate decay, five warmup epochs (twenty for the warmup probe), weight decay 0.05, gradient clipping at 1.0, label smoothing 0.1, Mixup 0.8, CutMix 1.0, RandAugment, and random erasing probability 0.25. The base learning rate is 5×10^{-4} ; a separate MergeNet arm uses 7.5×10^{-4} . The higher-rate arm supplies the 300-epoch MergeNet checkpoint. Therefore the final dense-versus-MergeNet comparison is a comparison of selected configurations, while the geometry comparison uses matched learning rates. No teacher distillation is enabled. Appendix A gives the recipe and auxiliary-logit schedule.

The effective global batch is 1,024 throughout the recorded campaign. The 384/512-pixel fine-tunes use four-step gradient accumulation: $32 \times 8 \times 4 = 1024$ for the dense 384-pixel run and $64 \times 4 \times 4 = 1024$ for the other fine-tunes. Each fine-tune epoch contains 5,004 per-rank microsteps and 1,251 optimizer steps. Counting microsteps as optimizer steps would incorrectly suggest a fourfold batch reduction. Different world sizes and microbatches still produce different execution and sample-grouping conditions; effective batch equality alone does not establish bitwise equivalence.

4.2 Resolution adaptation and checkpoint selection

Three 384-pixel runs fine-tune the corresponding 300-epoch EMA weights for 30 epochs, using learning rate 10^{-5} , minimum rate 10^{-6} , one warmup epoch, weight decay 10^{-8} , and no Mixup or CutMix. EMA decay changes from 0.99996 to 0.9996. The MergeNet arms either retain $R = 3$ or use $R = 3 \times 384/224 \approx 5.143$. The latter tests a particular proportional radius scaling; it is not a complete radius search. Bicubic validation preprocessing uses crop percentage 0.9 at 224 pixels and 1.0 for the fine-tunes. The completed 512-pixel dense result follows its 384-pixel checkpoint; there is no completed paired MergeNet result at that resolution.

We report both the best observed EMA top-1 and the final-epoch EMA top-1. Best checkpoints are selected on the same validation split used for reporting, so these scores are not independent held-out estimates of model selection. Epoch indices start at zero. All completed-run comparisons use actual summary headers rather than fixed column positions, because the summary schema changes with enabled diagnostics. Runs with a teardown error after a complete recorded schedule are distinguished from unfinished runs.

4.3 Post-training compression and timing

The accuracy sweep applies ToMe and PiToMe to the dense EMA checkpoint without additional training, and changes MergeNet’s carrier budget without retraining each point. Thus MergeNet has been trained

Routing / probe	LR (10^{-4})	Best (%)	Final (%)
Global	5.0	77.218	77.216
Flat window 8	5.0	78.290	78.278
Spatial $R = 2$	5.0	78.942	78.942
Spatial $R = 3$	5.0	78.888	78.888
Spatial $R = 3$	7.5	79.316	79.316
Spatial $R = 3$, warmup 20	7.5	79.238	79.238

Table 1: 224-pixel, 150-epoch routing controls and probes. The first four rows use the same learning rate and base architecture. The last two change optimization conditions; warmup 20 also uses a different per-rank microbatch. All accuracies are single-seed EMA top-1.

with its native compression, whereas the dense backbone was trained without merging. This is a deployment comparison of available checkpoints, not an isolation of the routing rule under identical training. We distinguish a 224-pixel checkpoint evaluated at a new resolution from a checkpoint fine-tuned at that resolution. Repeated successful sweep records are resolved by latest timestamp, not highest accuracy. All included accuracy cells use the full validation split. Training-summary validation uses 64 images per rank. For sweeps from the 224-pixel checkpoints, MergeNet and ToMe use batch 128 and PiToMe uses batch 256; the final 384-pixel fine-tuned curves use batch 256 for all three. The separate fine-tuned gate uses batch 384. These batch distinctions matter for the soft-routing implementation and are retained in the evidence ledger.

The 224-pixel efficiency experiment uses one H20-3e, synthetic inputs, batch 64, fp16 autocast, 30 warmup iterations and 50 measured iterations. It records CUDA-event medians and interquartile ranges (IQRs); training cells include forward, loss, backward and AdamW update. Device isolation is checked around the measurements. The separate radius-3 benchmarks at 384 pixels use the same thirty-warmup/fifty-measurement protocol at batches 64 and 128. These are model-execution timings, excluding image loading, and do not estimate end-to-end training time on eight GPUs. Training-log throughput is excluded because the training machine was shared.

A later 384-pixel benchmark uses batch 64, ten warmup and 30 measured iterations, random model initialization and synthetic inputs. It records achieved token counts, but disables ToMe proportional attention, whereas the accuracy sweep uses it. We retain these latency measurements as a separate configuration-specific experiment and do not attach accuracy coordinates to them. Final token counts also require care: the accuracy sweep counts patches, while this timing harness counts the class token as well.

5 ImageNet Results

5.1 Internal geometry controls

Table 1 separates routing controls from optimization probes. A separate historical CIFAR-100 study is reported in Appendix E, with its own resized-image and code-version boundaries. At learning rate 5×10^{-4} , the radius-3 model reaches 78.888%, versus 77.218% for global routing and 78.290% for flattened-window routing. The corresponding improvements are 1.670 *pp* and 0.598 *pp*. Their final-epoch differences are 1.672 *pp* and 0.610 *pp*, so best-checkpoint selection does not explain the ordering. Radius 2 reaches 78.942%, only 0.054 *pp* above radius 3. With one seed, this does not establish an optimal radius or statistical equivalence between the two choices.

The geometric result is specific to these support policies. Radius changes alter both spatial coverage and the number of eligible receivers. Since row energies sum over eligible receiver scores, candidate cardinality can affect donor selection even before transport. A degree-matched randomized support control would be needed to isolate geometric arrangement from this effect. Likewise, global MergeNet is not the original DTEM method: it shares MergeNet’s local front end, soft inference path, final gather, and recovery operation.

Raising the learning rate to 7.5×10^{-4} improves radius-3 accuracy by 0.428 *pp* at 150 epochs. Extending warmup from five to twenty epochs produces 79.238%, 0.078 *pp* below the higher-rate baseline. This is an observed endpoint difference, not evidence that longer warmup is generally ineffective. Besides being single-seed, the warmup run uses four ranks with 256 images per rank rather than eight ranks with 128. The batch-dependent operator discussed in Section 6.2 makes this execution difference material to

Model	Pixels	Epochs	Best (%)	Final (%)	Best epoch
DeiT-S/16	224	300	79.692	79.684	290
DeiT-S/8	224	300	82.246	82.162	277
MergeNet, $R = 3$	224	300	81.368	81.350	291
DeiT-S/8	384	300+30	82.976	82.848	7
MergeNet, $R = 3$	384	300+30	82.582	82.524	20
MergeNet, $R = 5.143$	384	300+30	82.574	82.532	22

Table 2: Complete main runs. Fine-tunes initialize from the corresponding 300-epoch EMA checkpoint. The selected MergeNet scratch configuration uses a higher learning rate than dense DeiT; the 384-pixel fine-tuning recipe is shared. Best and final checkpoints are kept distinct; fine-tuning best-epoch indices are relative to the fine-tuning stage.

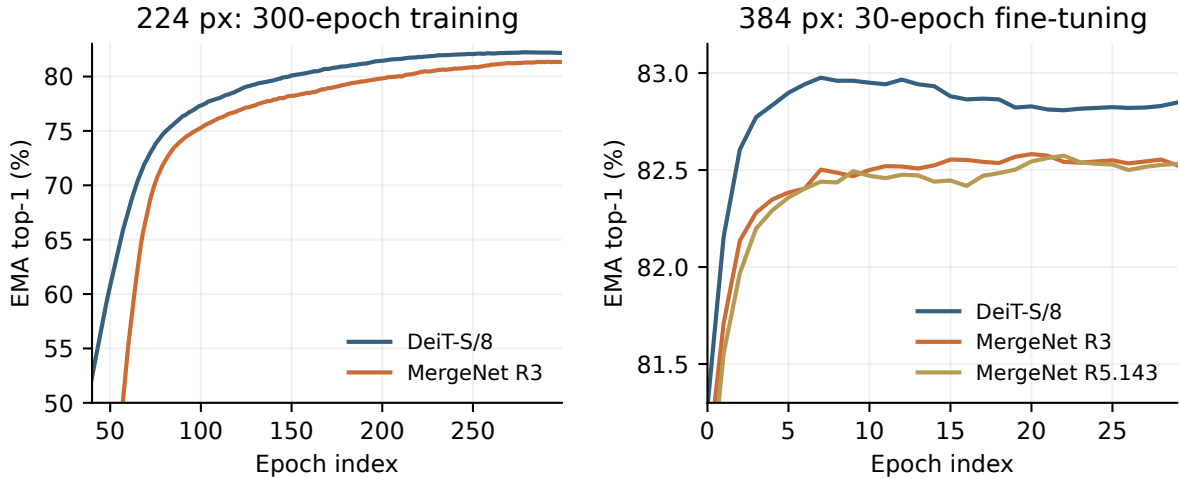


Figure 2: Recorded EMA validation curves. The left panel focuses on epochs 40–299 of complete 300-epoch runs; the right panel shows all 30 fine-tuning epochs. Curves share a seed and are not confidence intervals. Later peak epochs do not identify why adaptation occurs or predict a longer schedule.

a clean ablation. The unfinished compression-curriculum run is excluded from this table. A separate $\lambda = 4$ run reaches 78.366%, but also changes local/latent depths from 6/6 to 4/8 and the local window from 16 to 32; it cannot be interpreted as a compression-ratio-only experiment.

5.2 Complete schedules and resolution adaptation

Table 2 reports complete 300-epoch runs and 384-pixel fine-tunes. At 224 pixels, MergeNet trails dense DeiT-S/8 by 0.878 *pp* using best checkpoints and 0.812 *pp* using final checkpoints. It exceeds DeiT-S/16 by 1.676 *pp*, but the latter is a much cheaper model at this input resolution. The higher accuracy relative to coarse patches does not establish better efficiency.

After 30 epochs at 384 pixels, radius-3 MergeNet reaches 82.582%, compared with 82.976% for dense DeiT-S/8. Its best-checkpoint gain over its own 224-pixel result is 1.214 *pp*, while dense DeiT gains 0.730 *pp*. The gap therefore narrows to 0.394 *pp*; using final checkpoints it is 0.324 *pp*. Scaling the radius to 5.143 produces 82.574%, a difference of -0.008 *pp* from radius 3. This one comparison gives no evidence of a useful accuracy gain from that particular scaling rule, but cannot rule out other radii or recipes.

Figure 2 shows that the models reach their best 384-pixel scores at different epochs: 7 for dense DeiT, 20 for radius 3, and 22 for radius 5.143. These observations do not prove that the router requires three times as much adaptation, nor that truncating a 30-epoch schedule would reproduce a newly scheduled ten-epoch run. Similarly, short terminal slopes cannot justify extrapolating another 300 training epochs. The completed 512-pixel dense leg reaches 83.068%, but the packet has no completed MergeNet epoch at 512 pixels, so no paired conclusion is reported there.

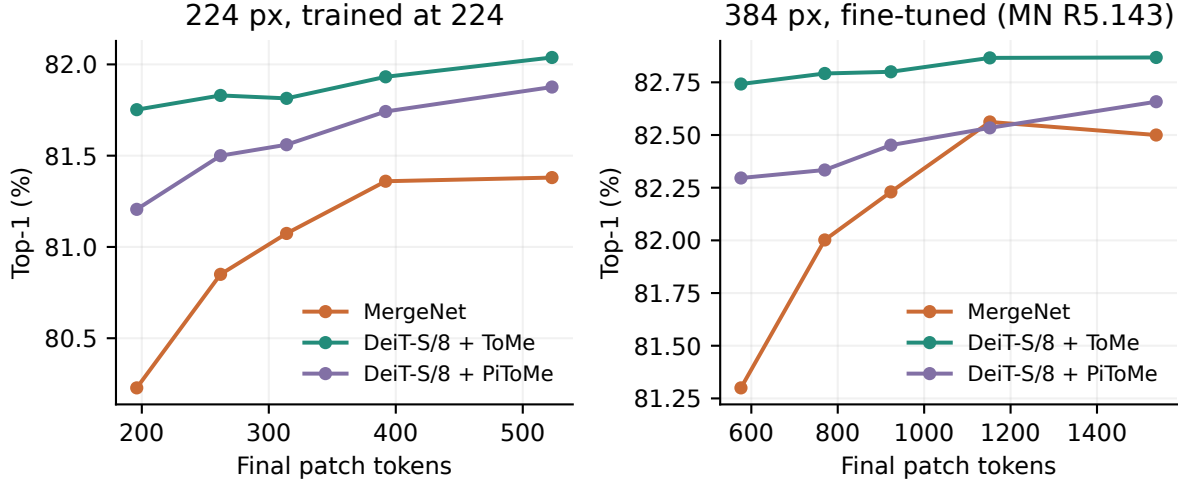


Figure 3: Accuracy-only budget sweeps. Left: 300-epoch 224-pixel checkpoints. Right: 384-pixel fine-tuned checkpoints, with radius 5.143 for MergeNet. ToMe and PiToMe require no additional training on top of the trained dense backbone. Lines connect measured points, not confidence bands. Equal final patch counts do not match the full computation trajectory.

5.3 Compression after training

Figure 3 compares full-validation accuracy at equal final patch counts. At 224 pixels and 392 retained patches, ToMe achieves 81.932%, compared with MergeNet’s re-evaluated 81.360%. At 196 patches, the scores are 81.752% and 80.228%, a 1.524 *pp* difference. The latter MergeNet point changes the budget of the $\lambda = 2$ checkpoint at evaluation; it is not the separately trained, structurally different $\lambda = 4$ model.

For 384-pixel fine-tuned checkpoints, ToMe reaches 82.866% at 1,152 patches, versus 82.562% for MergeNet, and 82.742% at 576 patches, versus 81.300%. Across all twenty matched budget points—five at native 224 pixels, five each at 288 and 384 pixels without resolution fine-tuning, and five after 384-pixel fine-tuning—ToMe has higher recorded accuracy. The differences range from 0.304 *pp* to 1.524 *pp*. These points are correlated evaluations of a small number of checkpoints, not twenty independent training trials. PiToMe is also competitive, but does not exceed MergeNet at every point; for example, at 1,152 patches after fine-tuning its 82.534% is slightly below MergeNet’s 82.562%.

Sweep values are re-evaluations and need not equal the training summary exactly. For example, the native 224-pixel MergeNet sweep records 81.360% versus a training best of 81.368%. We preserve both measurements rather than rounding them into agreement. Repeated PiToMe evaluations also differ by up to 0.034 *pp* in the supplied record. This is variation in repeated evaluation, not a measured across-seed training variance or a universal noise threshold. Most importantly, these accuracy results alone do not establish a joint accuracy–latency advantage; Section 6 explains the harness mismatch that prevents that conclusion.

6 Execution and Implementation Diagnostics

6.1 Arithmetic savings do not guarantee lower latency

The 224-pixel analytic estimate is 14.13 GMAC per image for MergeNet and 22.41 GMAC for dense DeiT-S/8, a 36.9% reduction. This estimate counts multiply–accumulates as one unit, includes projections, MLPs, attention matmuls, routing transport, and recovery, and does not assign MACs to sorting, indexing, copying, or threshold selection. It is an arithmetic accounting model rather than a measured instruction count. Figure 4 shows why a final carrier budget cannot describe all computation.

The guarded synthetic benchmark nevertheless measures MergeNet inference at 45.71 ms per batch, versus 42.68 ms for dense DeiT-S/8 (Table 3). MergeNet takes 7.1% more inference time and 3.5% more training-step time in this experiment. It also uses more peak allocated memory. DeiT-S/16 is substantially faster, reinforcing that the accuracy gain from retaining a finer input grid has a real execution cost.

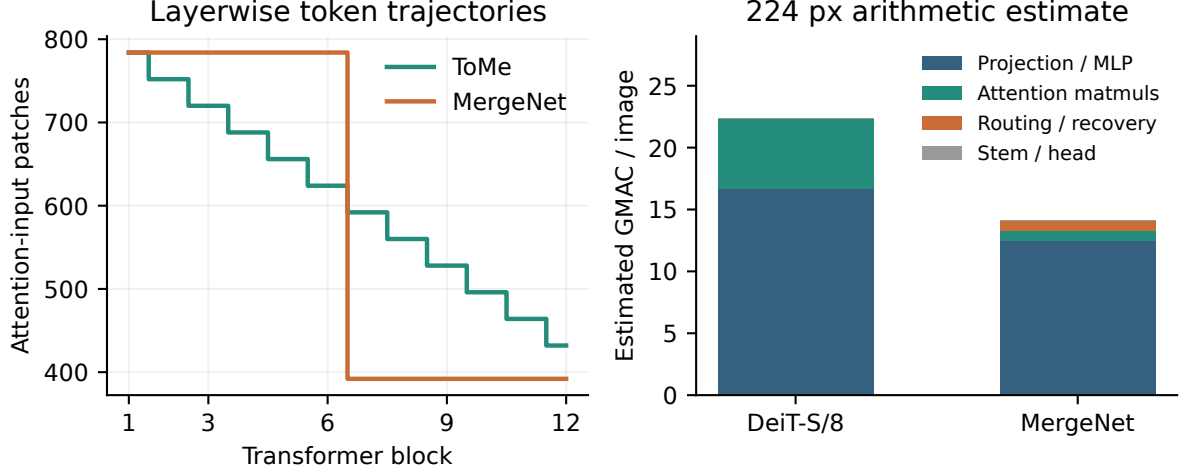


Figure 4: Two code-derived views of cost. Left: the attention-input patch trajectory at a final budget of 392; ToMe merges within blocks, while MergeNet physically reduces the sequence between stages. ToMe’s MLP inputs can be shorter than its attention inputs. Right: analytic MAC components, with non-arithmetic operations omitted from the count. Recovery is included in the MergeNet routing/recovery component. These are not measured GPU profiles.

Model	Params (M)	Train (ms)	Infer (ms)	IQR (ms)	Train GiB	Infer GiB
DeiT-S/16	22.05	37.62	9.99	0.005	2.472	0.367
DeiT-S/8	22.06	149.56	42.68	0.099	8.555	0.799
MergeNet, $R = 3$	23.10	154.87	45.71	0.073	10.071	1.125

Table 3: September 5, 2026, 224-pixel batch-64 synthetic H20-3e benchmark, with random model initialization and fp16 autocast. Train time includes forward, loss, backward and AdamW update. Infer time is a forward-only median; the IQR column refers to inference. Memory is peak *allocated* GiB. These model-execution cells are separate from the trained-checkpoint accuracy experiments.

Several implementation properties offer plausible explanations. Full-grid local MLPs and projections remain, sparse routing introduces indexed access and reductions, the threshold operator performs sorting and cumulative operations, and global recovery adds another attention operation. However, the delivered decomposition uses CPU operator profiles and analytic byte estimates. Its dispatch counts are not measured CUDA kernel-launch counts, and its logical tensor traffic is not measured HBM traffic. We therefore do not infer a proven memory-bandwidth bottleneck, a measured arithmetic intensity, or an end-to-end speedup from a proposed kernel optimization.

At 384 pixels, the guarded batch-64 radius-3 benchmark measures 136.54 ms for MergeNet inference and 170.61 ms for dense DeiT, a 20.0% reduction in batch time. A batch-128 run measures 262.02 versus 324.36 ms, a 19.2% reduction; its training-step times are 889.76 versus 1135.48 ms. Thus the measured cost balance changes with resolution. These architecture benchmarks use random initialization and remain distinct from checkpoint accuracy. A later radius-5.143 latency sweep measures ToMe with proportional attention *disabled*, unlike the accuracy sweep. It also uses slightly different achieved patch counts and retains token-counting hooks during timing. Appendix B reports these cells with their actual settings. The available record does not support attaching the earlier accuracy scores to these times or declaring that one method dominates the other on both axes. A common harness must use the same attention policy, actual budget, checkpoint identity, precision and hook-free timed path before making that claim.

6.2 Soft selection depends on batch composition

A separate CPU probe identifies a functional issue in the inherited threshold-based soft top- k implementation. For an input tensor $x \in \mathbb{R}^{B \times N}$, it normalizes using

$$\hat{x}_{bi} = 30 x_{bi} / \left(\max_{b',j} x_{b'j} - \min_{b',j} x_{b'j} \right). \quad (10)$$

The range is computed across the entire batch. Consequently, selecting weights for one sample can depend on unrelated samples. Holding the first row fixed at $(1, 2, 3, 4)$ and $k = 2$, changing only the second row from $(1.2, 2.2, 3.2, 3.8)$ to $(10, 20, 30, 40)$ changes a first-row soft weight by up to 0.336987. A loss depending only on the first row yields a nonzero gradient on the second row, with maximum magnitude 0.017355 in the probe. Both float32 and float64 exhibit the effect. An entirely constant batch also makes the unclamped denominator zero, producing non-finite outputs and gradients.

This operator is used directly in the routing assignment, so the issue is not limited to the final selection’s straight-through gradient. In particular, changing evaluation batch composition can change soft routing even in `eval` mode. The probe establishes the operator-level dependency; it does not establish how much any supplied ImageNet checkpoint’s top-1 would change, or explain the small sweep-versus-summary discrepancies by itself. The main same-recipe geometry controls remain comparisons under their recorded batch conditions. A corrected implementation requires its own checkpoint evaluation and training evidence rather than inheriting the reported scores without verification.

7 Discussion and Limitations

The results distinguish an internal architectural improvement from a competitive compression system. Original-grid support improves this architecture relative to its own unconstrained router. Yet the full model loses accuracy to the selected dense baseline, its 224-pixel implementation is slower in guarded synthetic tests, and post-training ToMe provides a stronger accuracy baseline at every matched budget in the sweep. These findings motivate revisiting the computation path and evaluation protocol; they do not show that learning a merge criterion is intrinsically inferior.

The evidence is limited in several ways. All training runs use seed 42, and the validation set serves both checkpoint selection and reporting. We have no across-seed uncertainty estimate or per-image predictions for paired significance tests. Geometry and candidate cardinality remain coupled. The 300-epoch MergeNet configuration benefits from an explicit learning-rate screen, while the dense baseline is not subjected to an equally broad screen. The $\lambda = 4$ training variant changes multiple architectural parameters, and test-time budget changes do not replace a clean training-budget ablation. The curriculum and paired 512-pixel result are incomplete at capture time.

The experiment artifacts also bound reproducibility. Raw metrics, resolved arguments, launch records, environment information, benchmark scripts and checkpoint receipts are available in the campaign packet. Actual trained weights were not transferred for this audit, and a release commit alone does not capture every change in the collaborator’s working tree. The companion analysis preserves the source identities and distinguishes recorded results from independent operator diagnostics. The training environment uses PyTorch 2.7.1 with CUDA 11.8, timm 0.9.11 and FlashAttention 2.7.3, which differs from the local release’s pinned environment. CPU profiles and H20 synthetic timings cannot establish performance on every accelerator or under deployment load. The ToMe attention-policy mismatch must be resolved before a joint frontier is reported.

The most informative next experiments follow directly from these limits. First, reevaluate transferred EMA weights with explicit batch composition and a common accuracy/timing configuration, recording exact per-layer shapes. Second, test a numerically stable, per-sample soft-selection operator and separate its effect from a change in training or evaluation batch. Third, repeat the global, flattened and radius-3 controls at additional seeds and include a degree-matched spatial control. System optimizations should then be evaluated with forward/backward parity and complete-model timing. These experiments would test concrete explanations for the observed gaps; merely extending a schedule based on a short terminal slope would not.

8 Conclusion

Spatially restricting soft token routing improves the recorded ImageNet accuracy of MergeNet relative to its global and flattened-window controls. The benefit survives both best- and final-checkpoint reporting, and higher-resolution fine-tuning narrows the gap to dense DeiT-S/8. It does not, however, establish a superior compression system. Full-grid processing before the gather, recovery cost, and routing overhead separate nominal compression from execution savings. The measured dense-relative latency balance improves at higher resolution, but strong post-training accuracy baselines and an attention-policy mismatch further limit deployment claims. The resulting study provides a reproducible account of the

measured tradeoffs and a specific batch-dependence diagnostic, while leaving a corrected, checkpoint-verified accuracy–latency frontier as an explicit open result.

References

- [1] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token Merging: Your ViT but Faster. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.09461v3>.
- [2] Mengzhao Chen, Wenqi Shao, Peng Xu, Mingbao Lin, Kaipeng Zhang, Fei Chao, Rongrong Ji, Yu Qiao, and Ping Luo. DiffRate: Differentiable Compression Rate for Efficient Vision Transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. URL <https://arxiv.org/abs/2305.17997v1>.
- [3] Wenyi Gong and Mieszko Lis. CubistMerge: Spatial-Preserving Token Merging for Diverse ViT Backbones, 2025. URL <https://arxiv.org/html/2509.21764v2>. arXiv version 2, March 2, 2026; first posted September 26, 2025.
- [4] Jung Hwan Heo, Seyedarmin Azizi, Arash Fayyazi, and Massoud Pedram. Training-Free Acceleration of ViTs with Delayed Spatial Merging, 2024. URL <https://arxiv.org/abs/2303.02331v2>. ICML 2024 ES-FoMo Workshop; arXiv version 2, July 1, 2024.
- [5] Rajat Koner, Gagan Jain, Prateek Jain, Volker Tresp, and Sujoy Paul. LookupViT: Compressing Visual Information to a Limited Number of Tokens. In *European Conference on Computer Vision*, 2024. URL <https://arxiv.org/html/2407.12753v1>.
- [6] Dong Hoon Lee and Seunghoon Hong. Learning to Merge Tokens via Decoupled Embedding for Efficient Vision Transformers. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/html/2412.10569v1>.
- [7] Sanghyeok Lee, Joonmyung Choi, and Hyunwoo J. Kim. Multi-criteria Token Fusion with One-step-ahead Attention for Efficient Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/papers/Lee_Multi-criteria_Token_Fusion_with_One-step-ahead_Attention_for_Efficient_Vision_Transformers_CVPR_2024_paper.pdf.
- [8] Yunge Li and Lanyu Xu. Neighbor-Aware Token Reduction via Hilbert Curve for Vision Transformers, 2025. URL <https://arxiv.org/html/2512.22760v1>. arXiv version 1, December 28, 2025.
- [9] Narges Norouzi, Svetlana Orlova, Daan de Geus, and Gijs Dubbelman. ALGM: Adaptive Local-then-Global Token Merging for Efficient Semantic Segmentation with Plain Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/papers/Norouzi_ALGM_Adaptive_Local-then-Global_Token_Merging_for_Efficient_Semantic_Segmentation_with_CVPR_2024_paper.pdf.
- [10] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. In *Advances in Neural Information Processing Systems*, 2021. URL https://papers.nips.cc/paper_files/paper/2021/file/747d3443e319a22747fbb873e8b2f9f2-Paper.pdf.
- [11] Quan Tang, Bowen Zhang, Jiajun Liu, Fagui Liu, and Yifan Liu. Dynamic Token Pruning in Plain Vision Transformers for Semantic Segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/papers/Tang_Dynamic_Token_Pruning_in_Plain_Vision_Transformers_for_Semantic_Segmentation_ICCV_2023_paper.pdf.
- [12] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 2021. URL <https://proceedings.mlr.press/v139/touvron21a.html>.
- [13] Hoai-Chau Tran, Duy M. H. Nguyen, Duy M. Nguyen, Trung-Tin Nguyen, Ngan Le, Pengtao Xie, Daniel Sonntag, James Y. Zou, Binh T. Nguyen, and Mathias Niepert. Accelerating Transformers with Spectrum-Preserving Token Merging. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.16148v2>.

- [14] Westlake-AI. OpenToMe: Toolbox and Benchmark for Token Merging Modules, 2025. URL <https://github.com/Westlake-AI/OpenToMe/tree/4b791cbc6bcef55040236c1852ff2f4102d82725>. Software; commit 4b791cbc6bcef55040236c1852ff2f4102d82725, August 12, 2026; upstream snapshot inspected September 11, 2026.
- [15] Weiying Xie, Xiaoyu Chen, Xin Zhang, Chenhe Hao, Jitao Ma, Yunsong Li, and Leyuan Fang. Saliency-Driven Token Merging for Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. URL https://openaccess.thecvf.com/content/CVPR2026/html/Xie_Saliency-Driven_Token_Merging_for_Vision_Transformers_CVPR_2026_paper.html.
- [16] Xuwei Xu, Sen Wang, Yudong Chen, Yanping Zheng, Zhewei Wei, and Jiajun Liu. GTP-ViT: Efficient Vision Transformers via Graph-Based Token Propagation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024. URL https://openaccess.thecvf.com/content/WACV2024/papers/Xu_GTP-ViT_Efficient_Vision_Transformers_via_Graph-Based_Token_Propagation_WACV_2024_paper.pdf.
- [17] Hongxu Yin, Arash Vahdat, Jose M. Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-ViT: Adaptive Tokens for Efficient Vision Transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/papers/Yin_A-ViT_Adaptive_Tokens_for_Efficient_Vision_Transformer_CVPR_2022_paper.pdf.

A Training Recipe and Evidence Inventory

Table 4 summarizes the resolved run arguments. The base width is 384 with six heads and MLP expansion four. Stochastic depth is configured at 0.1; MergeNet constructs separate local and latent ramps, unlike a single twelve-block dense ramp. The global and flattened routing controls use their dense/historical scoring paths, while radius-2/3 routing uses the sparse spatial backend; the routing support and its numerical implementation should therefore both be recorded. EMA validation replaces raw-model metrics before the summary row is written, so `eval_top1` is an EMA metric.

Setting	224-pixel scratch	384-pixel fine-tune
Epochs	150 or 300	30
Effective global batch	1024	1024
Optimizer	AdamW	AdamW
Learning rate	5×10^{-4} or 7.5×10^{-4}	10^{-5}
Minimum learning rate	10^{-5}	10^{-6}
Weight decay	0.05	10^{-8}
Warmup epochs	5 (probe: 20)	1
Warmup learning rate	10^{-6}	10^{-6}
EMA decay	0.99996	0.9996
Mixup / CutMix	0.8 / 1.0	0 / 0
Label smoothing	0.1	0.1
Random erasing	0.25	0.25
RandAugment	<code>rand-m9-mstd0.5-inc1</code>	same
Gradient clip norm	1.0	1.0
Validation crop percentage	0.9	1.0
MergeNet auxiliary-logit weight	ramp to 0.05	0.05
Teacher / feature / routing distillation	disabled	disabled

Table 4: Main recipe. Launch records take precedence over configuration-file defaults, especially for microbatch size and gradient accumulation.

The 150-epoch MergeNet runs start the auxiliary-logit contribution at epoch 10 and ramp it over 10 epochs; the 300-epoch run uses epoch 20 and a 20-epoch ramp. Fine-tunes enable weight 0.05 from the start. The recorded environment is Python 3.10.20, PyTorch 2.7.1+cu118, torchvision 0.22.1+cu118, timm 0.9.11, FlashAttention 2.7.3 and Triton 3.3.1 on H20-3e GPUs. Both four and eight ranks divide 50,000, avoiding validation-sampler padding for this split. Other world sizes require a separate check.

Three nominal launcher failures occurred after complete metric histories: the 150-epoch DeiT-S/16, the compound $\lambda = 4$ run, and dense 512-pixel fine-tuning. Complete summaries and supplied checkpoint receipts distinguish them from early failures. Receipts record checks on another machine, not weights independently loaded in this audit. The partial curriculum had reached $\lambda = 2$ by its latest recorded epoch, but its 64-epoch score cannot serve as a completed 150-epoch endpoint.

B Detailed Budget and Timing Comparisons

Table 6 gives the values behind the main accuracy panels. The MergeNet checkpoint uses radius 3 at 224 pixels and radius 5.143 after 384-pixel fine-tuning. Final patch budgets match within a row, but layerwise attention and MLP inputs do not. ToMe and PiToMe shorten sequences within successive blocks, while MergeNet gathers once between stages. The companion CSV also contains the two resolution-transfer panels using 224-pixel checkpoints without fine-tuning.

At the approximately half-size operating point, ToMe processes 1,151 patches and MergeNet 1,152, taking 129.810 and 155.255 ms respectively. These times do not prove a joint ordering for the accuracy sweep’s ToMe setting. A common harness should use exactly 1,152 patches with proportional attention enabled, verify the checkpoint and preprocessing, and remove shape hooks before timing. An earlier latency benchmark silently clamped several requested ToMe budgets to the same achieved count; those obsolete cells are excluded.

Separate 384-pixel radius-3 benchmarks remain distinct from the radius-5.143 sweep. Speed percentages at different batch sizes, radii or harness settings should not be averaged into an unexplained range. Peak reserved memory and peak allocated memory also measure different quantities.

Run ID	Epochs	Status	Best top-1
s2_deit_p16_150e	150/150	Complete	77.196
s2_deit_p8_150e	150/150	Complete	79.992
s2_mn_flat_w8_150e	150/150	Complete	78.290
s2_mn_global_150e	150/150	Complete	77.218
s2_mn_l4_150e	150/150	Complete	78.366
s2_mn_r2_150e	150/150	Complete	78.942
s2_mn_r3_150e	150/150	Complete	78.888
s2_mn_r3_lr75_150e	150/150	Complete	79.316
s3_deit_p16_300e	300/300	Complete	79.692
s3_deit_p8_300e	300/300	Complete	82.246
s3_mn_r3_lr75_300e	300/300	Complete	81.368
s3_deit_p8_ft384_30e	30/30	Complete	82.976
s3_mn_r3_ft384_30e	30/30	Complete	82.582
s3_mn_r5_ft384_30e	30/30	Complete	82.574
s4_mn_lamcurr_150e	64/150	Partial	63.592
s4_mn_warm20_150e	150/150	Complete	79.238
s4_deit_ft512_15e	15/15	Complete	83.068
s4_mn_ft512_15e	0/15	No epoch	–

Table 5: Status at packet capture. Partial metrics are included for accounting only. The companion data retain full precision and source summary lines.

Checkpoint / pixels	Patches	MergeNet	ToMe	PiToMe	Δ ToMe
224 trained / 224	523	81.380	82.038	81.876	+0.658
224 trained / 224	392	81.360	81.932	81.742	+0.572
224 trained / 224	314	81.074	81.814	81.560	+0.740
224 trained / 224	262	80.850	81.830	81.500	+0.980
224 trained / 224	196	80.228	81.752	81.206	+1.524
384 tuned / 384	1537	82.500	82.868	82.658	+0.368
384 tuned / 384	1152	82.562	82.866	82.534	+0.304
384 tuned / 384	923	82.230	82.800	82.452	+0.570
384 tuned / 384	770	82.002	82.792	82.334	+0.790
384 tuned / 384	576	81.300	82.742	82.296	+1.442

Table 6: Top-1 percentages at equal final patch counts; Δ is ToMe minus MergeNet in percentage points. These values cannot be paired with Table 7 to construct an accuracy–latency frontier.

C Operator-Level Diagnostic and Reproduction Boundaries

The batch-dependence probe imports only the released threshold-selection function. It does not instantiate an ImageNet model or load a checkpoint. A fixed first row is evaluated alone and with controlled second rows; a weighted loss on the first row tests cross-sample gradients. Constant-input cases test finite forward and backward behavior. These probes establish operator properties, not an accuracy gain from replacing the operator.

Per-sample normalization removes the demonstrated coupling, but does not by itself validate the closed-form threshold solver over every budget and tie pattern. A patched closed-form candidate passes 116 of 120 tensor coverage cases and is not recommended for integration. A separate slow reference finds a per-row threshold t for normalized scores z satisfying

$$\sum_i h(z_i - t) = k, \quad h(\delta) = \begin{cases} 1 - \frac{1}{2} \exp(-\delta), & \delta > 0, \\ \frac{1}{2} \exp(\delta), & \delta \leq 0. \end{cases} \quad (11)$$

For $0 < k < N$, the sum is continuous and strictly decreasing in t . With $v_i = \partial \mathcal{L} / \partial h_i$, implicit differentiation gives

$$\nabla_{z_i} \mathcal{L} = h'_i \left(v_i - \frac{\sum_j v_j h'_j}{\sum_j h'_j} \right). \quad (12)$$

PyTorch then propagates gradients through the per-row normalization to the raw scores. The reference normalizes per row, solves in float64 with 60 bisection steps, and casts the result back to the input dtype. It passes all 120 coverage cases for finite forward/backward values and selection mass, plus three nondegenerate gradient checks. Maximum mass error is 9.16×10^{-5} in float32 and 1.14×10^{-13} in

ToMe patches	MN patches	ToMe ms	MN ms	ToMe GiB	MN GiB
1536	1538	147.30	178.17	2.793	3.619
1151	1152	129.81	155.25	2.764	3.564
922	923	122.44	143.02	2.746	3.721
769	770	118.17	136.59	2.738	3.719
575	576	106.45	129.38	2.723	3.557

Table 7: Separate 384-pixel synthetic latency experiment: batch 64, H20-3e, fp16 autocast, random initialization, MergeNet radius 5.143, and ToMe proportional attention disabled. Patch counts exclude the class token here. Memory is peak *reserved* GiB, unlike Table 3. Token-counting hooks remain active during timing.

float64; the tested cross-sample output and gradient dependencies are zero. These tests cover random, constant, nearly constant and outlier inputs across $N \in \{4, 16, 64, 392\}$ and several budgets. This is a correctness reference, not a fused production kernel or a trained replacement for the reported model. Finite gradients can still be large near a zero score range; GPU/autocast and full-model stability were not evaluated. Mass tolerances in these checks are 5×10^{-4} for float32 and 10^{-6} for float64.

The artifact subset includes unmodified per-run summaries, resolved arguments with host paths removed, launch-protocol extracts, sanitized successful sweep records, audited benchmark tables, and source hashes. The CPU rebuild script recomputes best/final metrics, resolves sweep records by timestamp, checks numeric joins, and regenerates tables and figures. It never launches training. Source and claim ledgers record the literature cutoff of 2026-09-15. The complete packet was inspected; trained weights still require transfer for independent re-evaluation.

D A Bounded Layout Optimization Experiment

We additionally test a single LocalAttention module on an A100-SXM4-80GB. The original module constructs query/key/value tensors in batch-head-token-channel order, then transposes and copies them for the FlashAttention wrapper. An isolated variant constructs batch-token-head-channel tensors directly and adjusts the global class-query attention views and final reshape accordingly. It does not alter the released model, load a checkpoint, or apply an optimizer update.

The test uses PyTorch 2.6.0+cu124 and FlashAttention 2.7.4.post1, width 384, six heads, window radius 16, zero attention dropout, float32 parameters and inputs, and fp16 autocast. Four batch-2 parity cases cover sequence lengths 785 and 2,305, global class attention on/off, and query/key normalization. Outputs and input gradients are bitwise equal in these cases. Parameter gradients are also equal except for the normalization case, whose maximum relative L_2 error is 5.77×10^{-7} . This coverage does not establish all-shape or dropout/RNG-preserving equivalence for a complete model.

Batch	Tokens	Mode	Original ms	Layout ms	Reduction range
64	785	Forward	1.5600	1.0448	13.50–34.03%
64	785	Forward + backward	5.1590	4.3999	14.68–25.64%
16	2305	Forward	1.0318	0.9369	6.88–36.68%
16	2305	Forward + backward	5.3740	4.9176	5.45–11.83%

Table 8: Single-module A100 measurements. Times are medians of three round medians, each with five warmups and twenty CUDA-event samples. The final column is the range of paired-round time reductions, not a confidence interval. Tokens include the class token. Backward includes a squared-output loss and gradient clearing, with no optimizer.

Measurement order alternates by round, and the harness requires an initially empty device, establishes its CUDA process identity, then rejects a changed process set at subsequent checks. All iteration samples and snapshots are retained. The observed reductions support this local engineering direction, but the round variation is appreciable. Both module instances are resident, so their recorded memory peaks are not independent per-model memory measurements. These results do not predict the full model’s speedup, validate an HBM-traffic estimate, transfer H20 timings to A100, or resolve the separate ToMe attention-policy mismatch.

E Historical CIFAR-100 spatial-routing ablation

We retain a historical CIFAR-100 experiment as a limited diagnostic of the routing geometry. The August campaign contains 60 new runs: two MergeNet configurations, five input sizes, and six routing settings (Global and Euclidean radii $R \in \{1, 2, 3, 4, 8\}$), all using seed 42. The configurations have local/latent depths of 6/6 with final compression ratio $\lambda = 2$, or 4/8 with $\lambda = 4$; both use width 384, six heads, and 8×8 patches. Each model was trained from scratch for 200 epochs without distillation. Training used AdamW, learning rate 10^{-3} , weight decay 0.05, a cosine schedule with 20 warmup epochs, effective batch size 200, and EMA decay 0.9998. The primary endpoint is the final epoch-199 EMA top-1 accuracy on the 10,000-image CIFAR-100 test set. The frozen local campaign protocol designated $R = 3$ as the primary setting before the recorded training launches; this is an internal protocol record, not an external preregistration. We keep that setting even though $R = 2$ has a slightly higher mean.

The original images are 32×32 pixels and are resized to the listed input sizes. Thus, this sweep changes the patch grid over upsampled images and does not test native high-resolution image content. Evaluation used FP16 autocast and deterministic fast grouping, with batches of 200, 200, 200, 100, and 50 at sizes 160, 192, 224, 256, and 320, respectively. Global and the finite-radius settings were trained in the same frozen implementation and share the non-routing configuration. The additional Flat reference uses the historical flattened-window implementation with width parameter 8 and comes from ten earlier August runs. Its recorded non-routing training arguments and dependency versions match, but its runtime code snapshot differs; three Flat runs resumed within their original jobs. We therefore use Flat as a historical descriptive reference.

Configuration	Input	Global	Flat [†]	$R = 3$	ΔG	ΔF
6/6, $\lambda = 2$	160	67.80	68.79	69.42	+1.62	+0.63
	192	69.42	69.93	70.85	+1.43	+0.92
	224	69.50	70.53	70.53	+1.03	+0.00
	256	69.50	70.12	71.03	+1.53	+0.91
	320	69.85	70.81	71.42	+1.57	+0.61
4/8, $\lambda = 4$	160	65.29	66.69	67.92	+2.63	+1.23
	192	69.87	69.75	71.27	+1.40	+1.52
	224	68.21	69.87	70.52	+2.31	+0.65
	256	67.35	68.30	69.40	+2.05	+1.10
	320	66.05	67.08	68.83	+2.78	+1.75
Mean		68.284	69.187	70.119	+1.835	+0.932

Table 9: Historical CIFAR-100 final-epoch EMA accuracy (%) and $R = 3$ deltas (percentage points) against Global (ΔG) and Flat (ΔF). Each row uses seed 42; the mean gives equal weight to the ten configuration–input pairs. [†]Flat is a historical reference with a different code snapshot. Input denotes the resized side length of native 32×32 images. These are observations under the stated evaluation batches and historical dense-score implementation.

At the fixed $R = 3$ setting, accuracy exceeds Global in all ten pairs, with a mean improvement of 1.835 percentage points. Relative to historical Flat, nine pairs improve and one ties, with a mean difference of 0.932 points. Across the complete sweep, the mean accuracies for Global and radii 1, 2, 3, 4, and 8 are 68.284, 68.774, 70.225, 70.119, 70.047, and 68.414%, respectively. Radii 2, 3, and 4 each exceed their matched Global control in all ten pairs. These correlated, single-seed observations provide evidence that restricting routing neighborhoods can help in this particular setting; they do not establish statistical significance, native-resolution transfer, or superiority over external token-reduction methods.

The audit reconciled all 60 new endpoints and the ten Flat endpoints with raw 200-epoch summaries, completion receipts, and frozen manifests. Archived full-test evaluations of the 60 new epoch-199 EMA checkpoints show identical generic- and fast-grouping logits; their accuracies differ from the training summaries in 31 cases by at most 0.02 points. The table preserves the registered training-summary endpoint. This audit checked receipt hashes and checkpoint presence, without a fresh checkpoint evaluation or full checkpoint rehash. The historical spatial implementation constructs a dense donor–receiver score matrix before masking, so these results do not validate the later sparse backend or its efficiency. Furthermore, both historical implementations use a threshold operator whose normalization range is shared across a batch, allowing a sample’s routing to depend on its batch companions. The effect on these checkpoints has not been quantified; the reported comparisons apply to the recorded evaluation batches.